

악성 URL 통합관리 서비스

인터넷을 안전하게

피싱 박스



AI 보안 기술개발 네트워크반 6조 최종
보고



목차

1. 팀원 소개
2. 문제의식 및 기획 의도
3. 프로젝트 소개
4. 시연
5. 프로젝트 상세 내용
6. 인공지능 모델 상세
7. 향후 발전 계획



팀원 소개



강세림
프론트엔드,
플러그인
데이터 전처리
인공지능-로지스틱



김용호
프론트엔드,
플러그인
데이터 전처리
인공지능-로지스틱



강인석
백엔드-인공지능 서버
인공지능-DNN



신동범
플러그인
인공지능 총괄

신동호
백엔드-웹 서버
인공지능-DNN

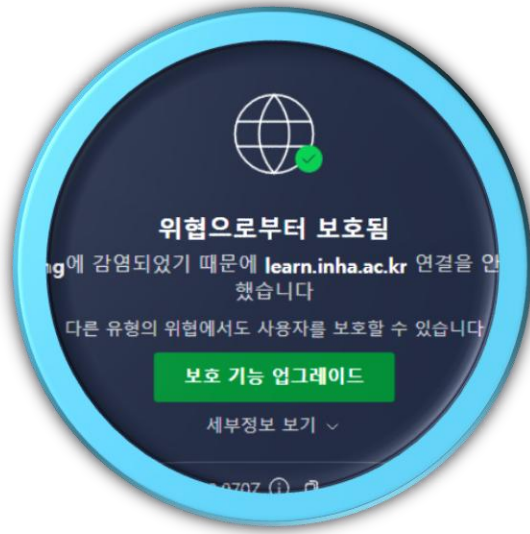


문제의식 및 기획 의도





문제의식 및 기획 의도



- ▶ 수많은 악성 사이트
- ▶ 한눈에 구분 불가능

- ▶ 데이터베이스 구축보다 빠르게 확산
- ▶ 오진으로 인한 피해

- ▶ 인공지능을 사용한 자가 진단



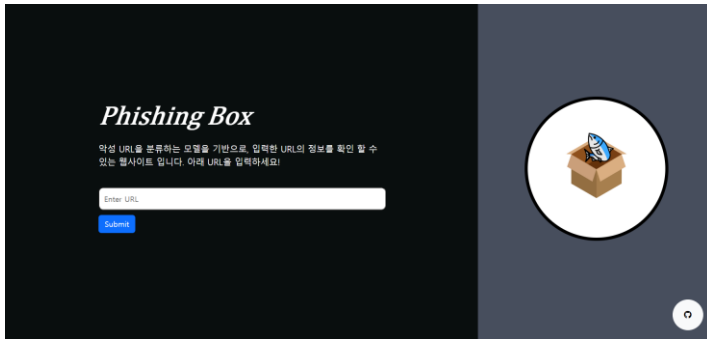
주제

- ▶ 인터넷을 안전하게 사용할 수 있도록 안전한 URL 진단 및 결과를 전달

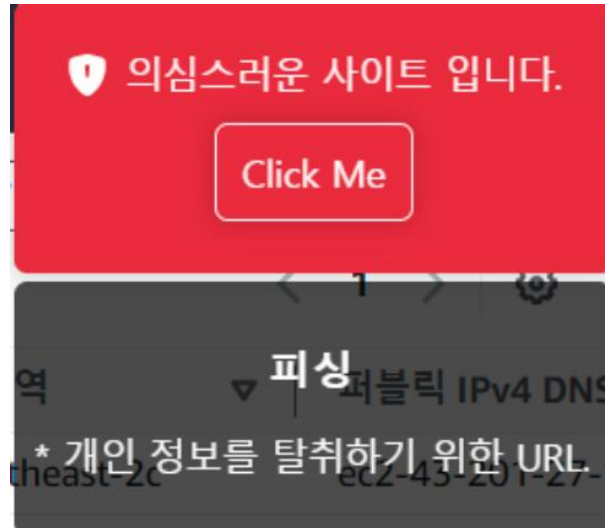




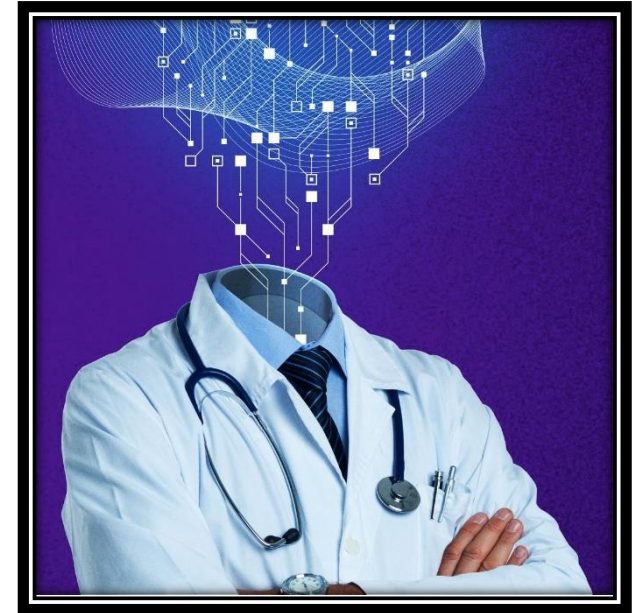
주요 기능



웹 페이지
의심스러운 URL 진단 및 상세
정보 확인 가능



크롬 플러그인(확장 프로그램)
크롬 브라우저 내부에서 접속하는
URL의 악성 여부 진단 및 경고



인공지능 진단
추가적인 데이터베이스의
업데이트 없이도 새로
발생하는 악성URL 판별



프로젝트 진행 현황표

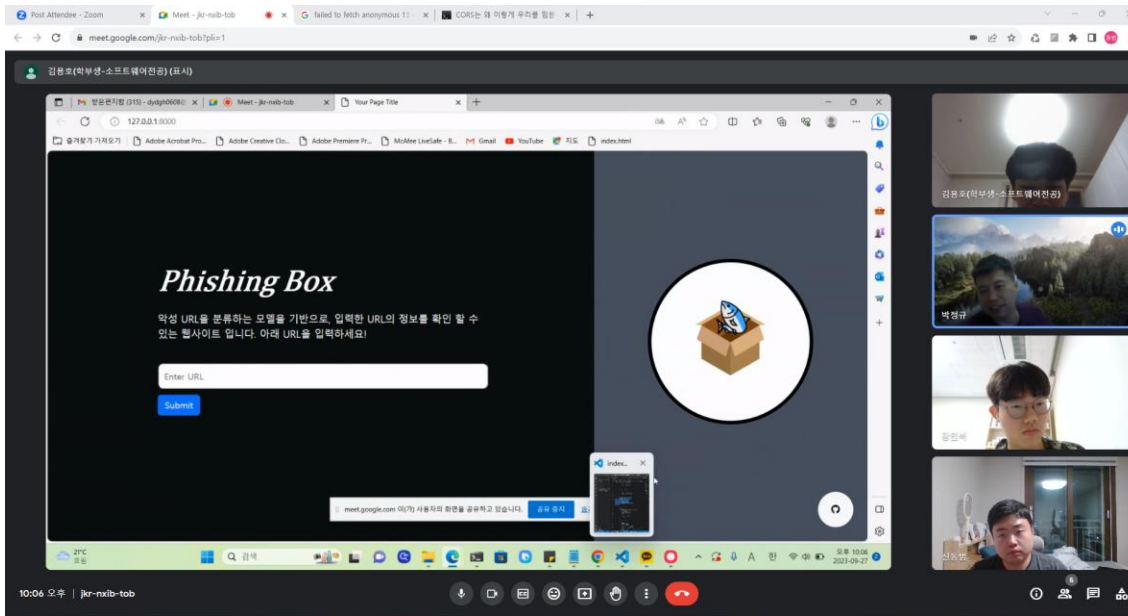
중간 보고회

최종 보고회

7월	8월				9월				10월			
프로젝트 주제 결정												
		기획, 역할 분담										
			백엔드	서버 개발-Django, FastAPI, NGINX, uWSGI								
			머신러닝	모델 개발-Logistic Regression, Pytorch DNN								
			웹 사이트	프론트엔드 개발								
					구글 크롬 플러그인(확장 프로그램)개발							
										통합 테스트		



프로젝트 진행



정보를 저장하는 모델 만들기 #20

merged yonghoya merged 4 commits into master from db_model_setting on Sep 10

Conversation 2 Commits 4 Checks 0 Files changed 3

yonghoya commented on Sep 6

개요

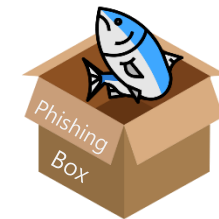
- SQLite를 사용하여 url 정보를 저장하는 모델 구현
- SQLite는 개발 단계에서 사용할 계획이며, 서비스로 제공할 때는 다른 DB로 바꿀 예정

변경 사항

1. URL 정보(type, type 설명, 악성 유무) 모델 작성
2. makemigration, migrate로 테이블 작업 파일과 테이블 생성
3. db.sqlite3에 URL 정보 데이터 생성 및 저장

☐	web_backend_fix feature fix	#18 by schsdh was merged on Sep 3
☐	this_is_test	#17 by schsdh was closed on Sep 3
☐	this_is_test	#16 by schsdh was closed on Sep 3
☐	PR_1	#15 by schsdh was closed on Sep 3
☐	backend build feature	#14 by schsdh was closed on Sep 3
☐	test	#13 by schsdh was closed on Aug 31
☐	branch_test	#12 by schsdh was closed on Sep 3
☐	method 변경 및 폼 크기 수정 fix	#11 by serimi was merged on Sep 3
☐	output page 수정 fix	#10 by yonghoya was merged on Sep 3
☐	Ai model	#9 by DongBoumShin was merged on Sep 3
☐	Revert "Ai model" fix	#8 by yonghoya was merged on Aug 30
☐	Ai model	#7 by DongBoumShin was merged on Aug 30

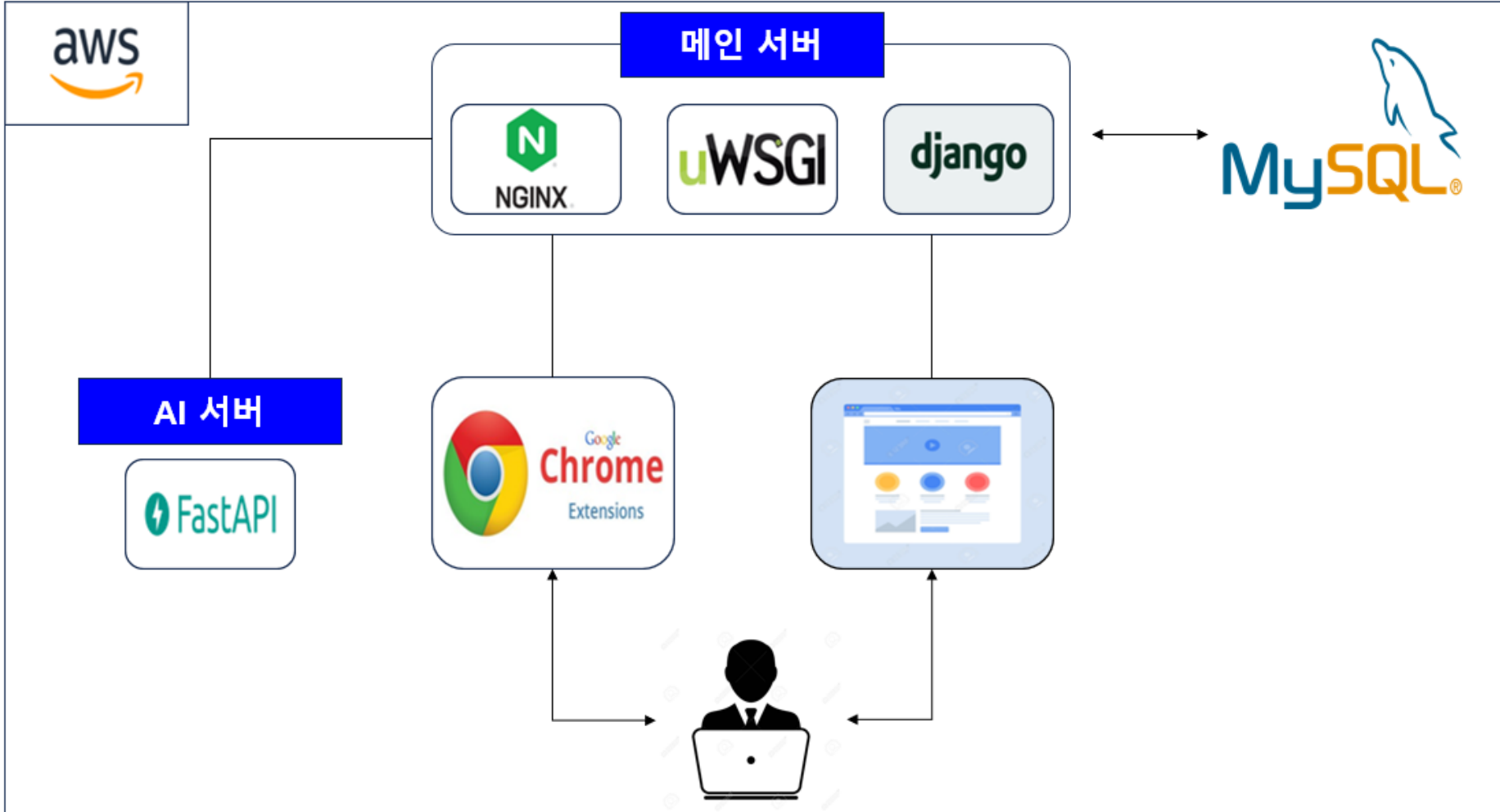
1. 문제의식 및 기획의도 2. 프로젝트 소개 3. 프로젝트 상세 내용 4. 인공지능 모델 상세 5.향후 계획



시연

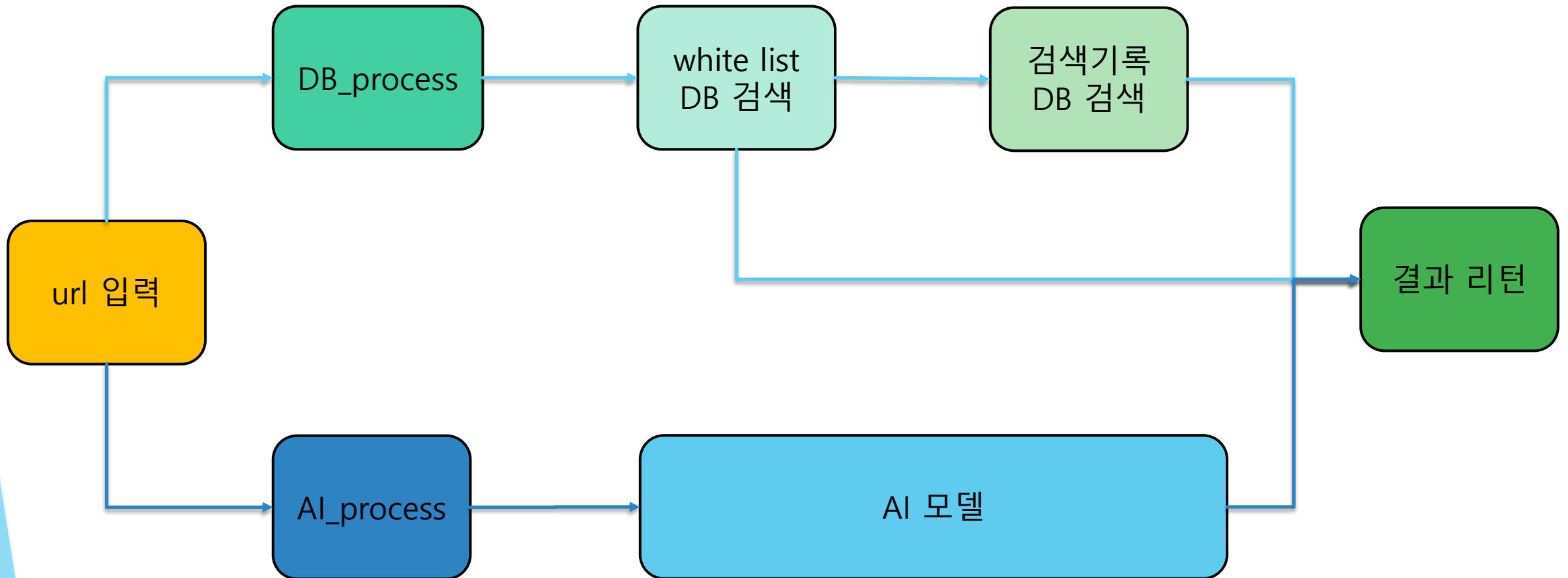


전체 서비스 개요





Backend-DB Multiprocessing





Backend-AI Server

- ▶ AWS EC2 인스턴스
- ▶ FastAPI로 AI모델 서버 구축
- ▶ AI모델이 무거워 빠른 요청/응답에 대해서 Django, Flask에 비해 가장 처리속도가 빠른 FastAPI로 구축함.





Backend-AI Server

- ▶ Nginx기반 웹 서버를 구축하여 Apache 웹 서버에 비해 빠르고 가볍기 때문에 안정적인 서비스 제공
- ▶ Django와 통신을 위해 uwsgi를 사용하여 원활하게 Nginx와 연결.





Backend-MySQL

▶ Django AWS에 MySQL을 설치하여 로컬환경에서 구축

- 예측결과값, URL, IP, 국가정보를 저장하는 테이블

결과값을 저장해두면서 같은 URL에 요청을 받을 경우 해당값을 반환해줌.

- white_list DB정보를 저장하는 테이블

잘 알려진 URL에 대해서 저장해두면서 URL이 같은 경우 AI처리를 거치지 않고 정상값 반환.

Field	Type	Null	Key	Default	Extra
prediction_result	tinyint(1)	NO		NULL	
prediction_type	varchar(100)	NO		NULL	
url	varchar(100)	NO	PRI	NULL	
ip_addr	varchar(50)	YES		NULL	
nation_code	varchar(50)	YES		NULL	

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
url	varchar(100)	NO		NULL	



웹 애플리케이션-Output.html

- 입력된 URL 악성 여부 및 정보 반환 페이지

- 반환되는 정보

1. URL
2. 악성 여부 분석 결과
3. 데이터 타입
4. 타입 설명
5. IP 주소
6. 국가
7. 국가 이미지

The screenshot displays the 'Phishing Box' web application. The main content area is titled 'URL 정보' (URL Information) and contains several sections:

- URL:** A text box containing 'www.naver.com'.
- 분석 결과 (Analysis Result):** A section with a green checkmark icon and the text '안전한 URL 입니다.' (It is a safe URL).
- 타입 (Type):** A section with a green checkmark icon and the text '정상' (Normal). A button labeled 'Click to view type description' is present.
- IP 주소 (IP Address):** A text box containing '223.130.195.200'.
- 국가 (추정) (Country (Estimated)):** A text box containing 'South Korea'. A button labeled 'Click to view image' is present.

At the bottom of the main content area, there is a button labeled 'Back to Input'.

On the right side of the interface, there are two pop-up boxes:

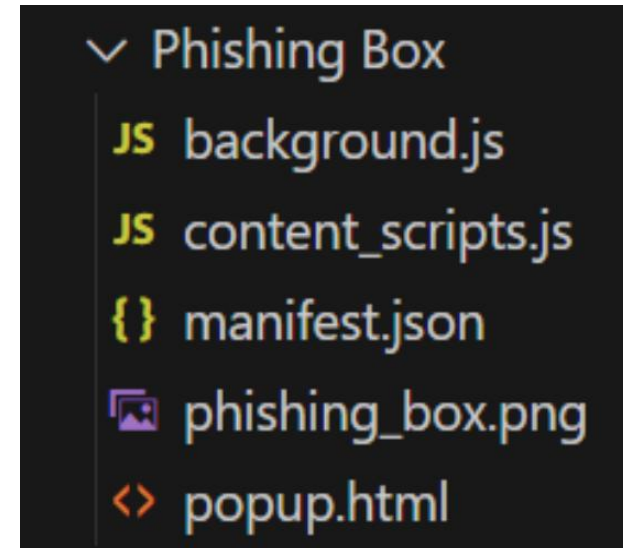
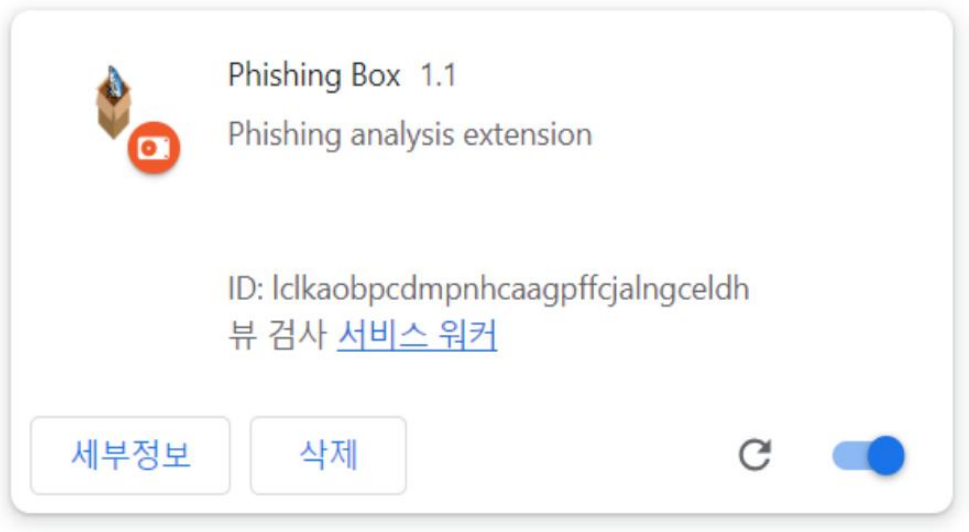
- A box titled '정상' (Normal) with the text: '악의적인 목적이나 위험을 갖지 않는 일반적인 웹 페이지 주소입니다.' (It is a general web page address that does not have malicious intent or risk).
- A box titled 'South Korea' with the South Korean flag image.

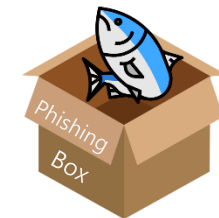
The footer of the application reads 'Phishing Box Made By GitHub'.



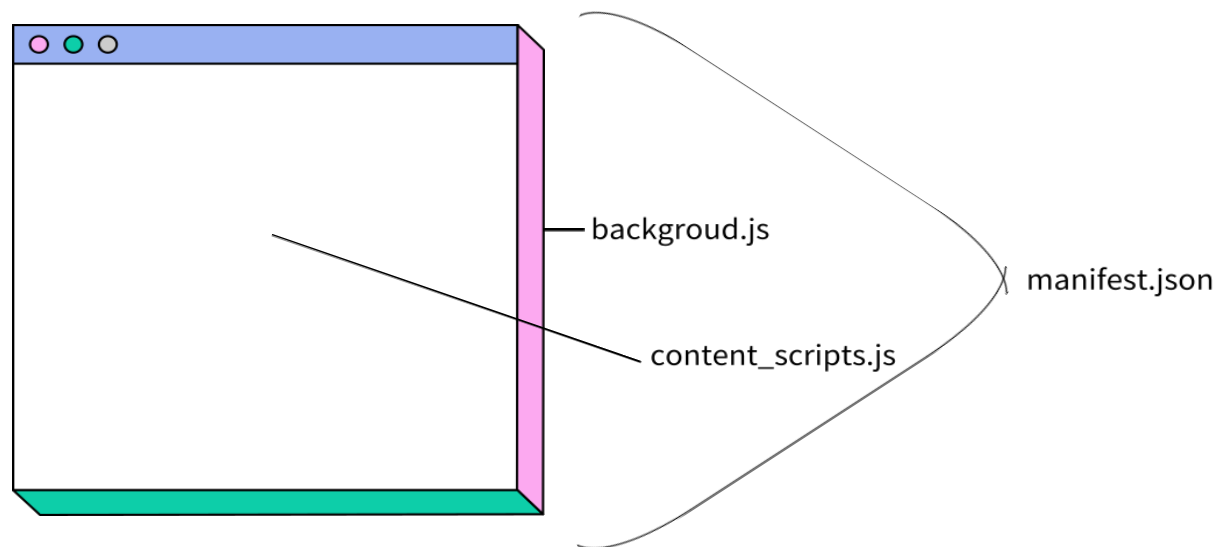
크롬 확장 플러그인

확장 프로그램





크롬 확장 플러그인-구성도



content_scripts.js

- 현재 URL 확인
- 결과에 따른 정보 제공
- 시각적 표시

content_scripts.js

- content_script 메시지 처리
- URL 로컬 스토리지 검색 or 서버로부터 데이터 가져와 저장
- 결과 반환

manifest.json

- manifest를 통해
Chrome 브라우저에서
확장 프로그램 동작



크롬 확장 플러그인-시각화 및 UI

benign: 안전한 사이트일 경우

```
switch (response.results) {  
  case "benign"  
    const allowedDiv = createDraggableCard('#19C37D', '');
```

녹색 카드 생성
해당 내용 표시

SERVER ERROR: 서버 오류가 발생한 경우

```
case "SERVER ERROR":  
  const errorDiv = createDraggableCard('yellow', 'Server ERROR');
```

노란색 카드 생성
에러 메시지 표시

의심스러운 사이트일 경우

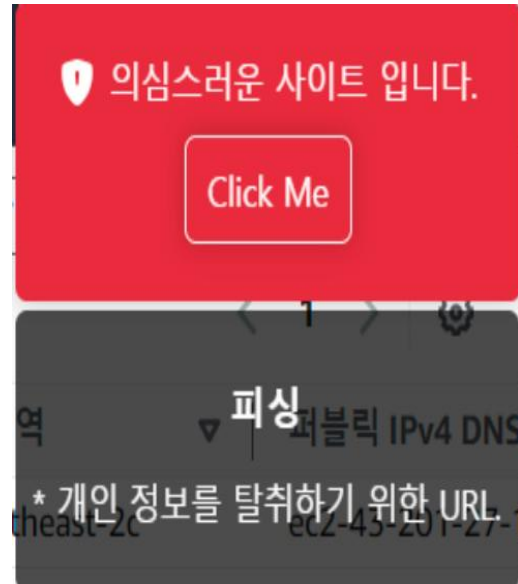
```
default:  
  const blockedDiv = createDraggableCard("#EA2A3D", '');
```

빨간색 카드 생성
해당 내용, 버튼 표시



크롬 확장 플러그인-시각화 및 UI

의심스러운 사이트일 경우



- Click Me 클릭
- 팝오버
- URL분류 정보 표시



인공지능

- ▶ Feature Extraction using Heuristics, BERT
- ▶ Logistic Regression



인공지능-Data

- ▶ Kaggle 'Malicious URLs Dataset'
 - ▶ <https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset>
- ▶ 651,191 urls
 - ▶ 428103 benign, 96457 defacement, 94111 phishing, 32520 malware
- ▶ Colab 환경 제한 상 전체의 15%, 약 96000개의 URL만 사용

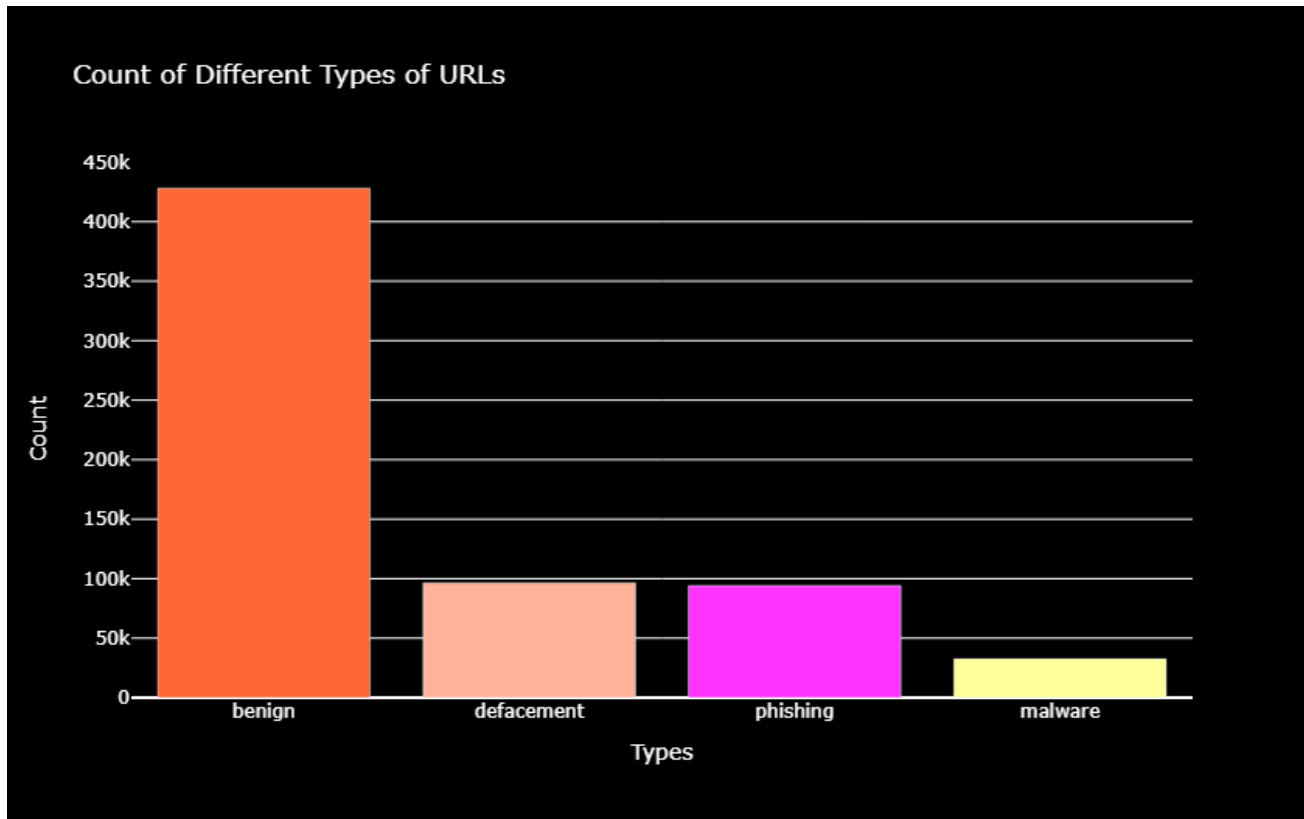


인공지능-Data

- ▶ 데이터셋 내부의 문제
 - ▶ 원 저작자가 규합 시 아래 사이트의 데이터를 반대로 저장함
<https://research.aalto.fi/en/datasets/phishstorm-phishing-legitimate-url-dataset>
- ▶ 위 사이트에서 데이터를 받아 데이터셋을 수정함



인공지능-Data



- ## 5.향후 계획

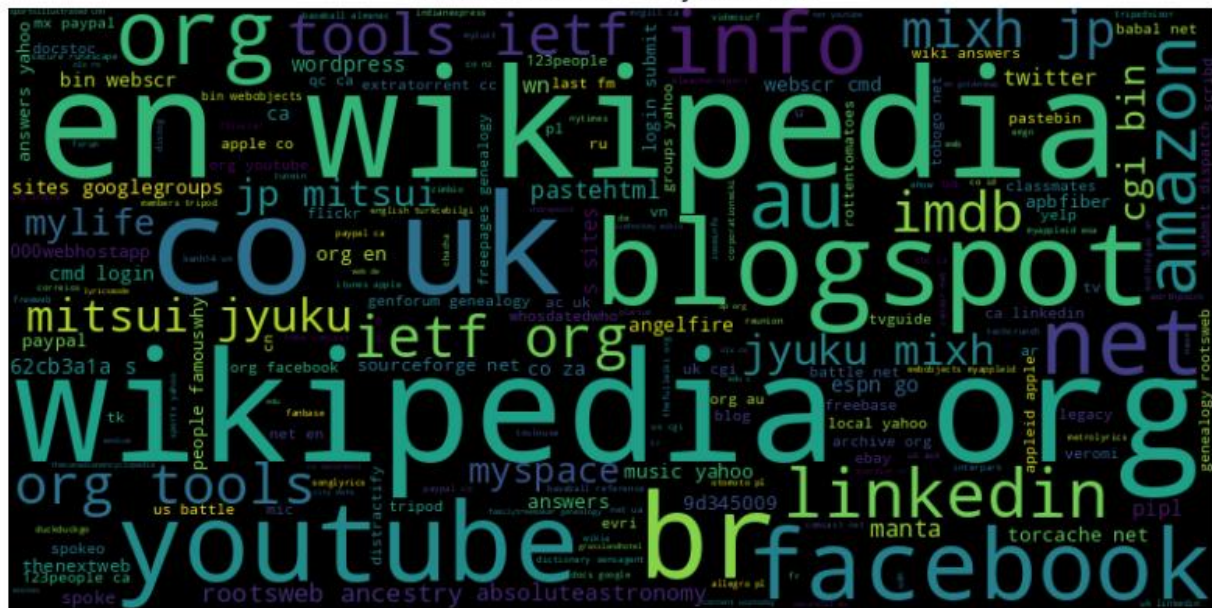


인공지능-Data

Word Cloud of URLs



Word Cloud of Primary Domains





Preprocessing

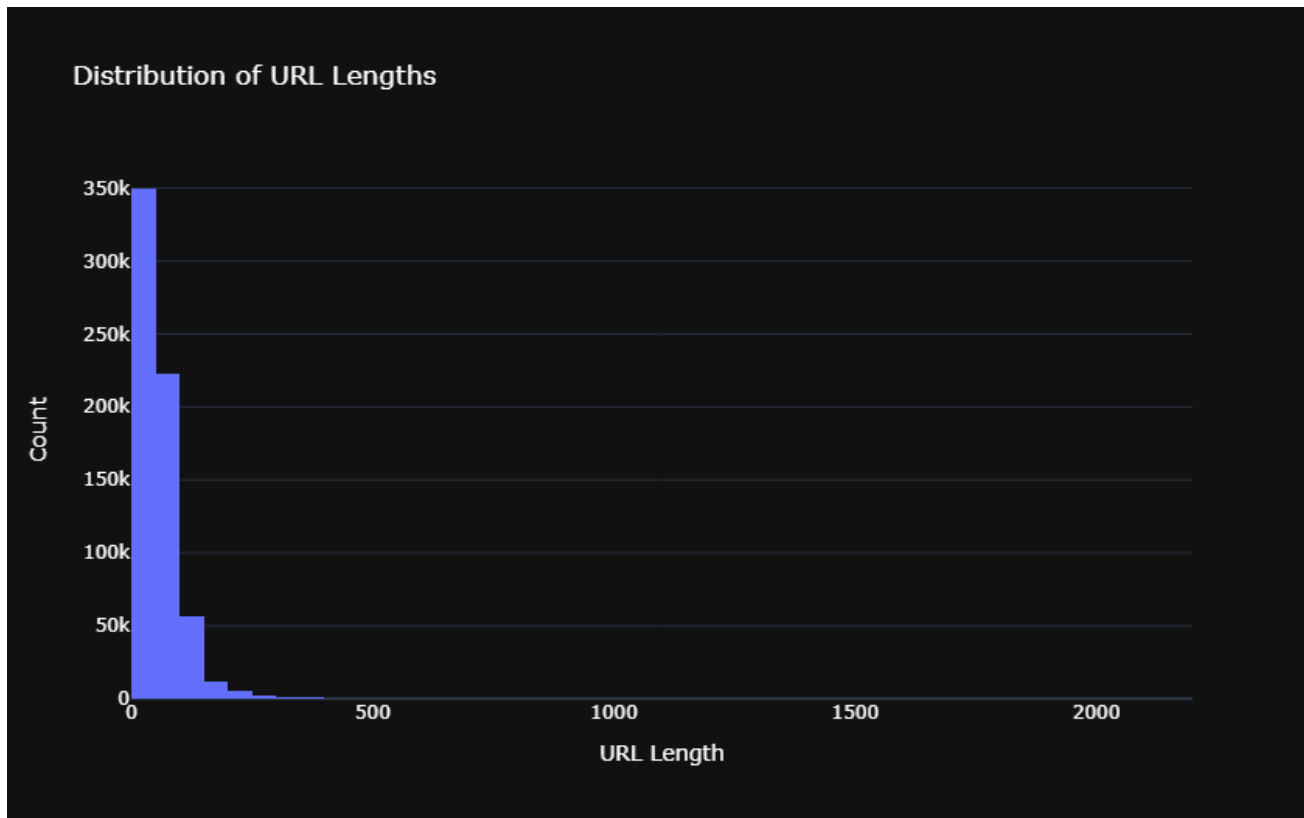
▶ Features:

- ▶ URL 길이
- ▶ 문자, 숫자, 특수문자 수
- ▶ 특수한 도메인
 - ▶ Hostname=Domain인 경우
 - ▶ 링크 단축 서비스 여부
 - ▶ IP주소 여부
- ▶ BERT model Hidden Layer



Preprocessing-Data Visuals

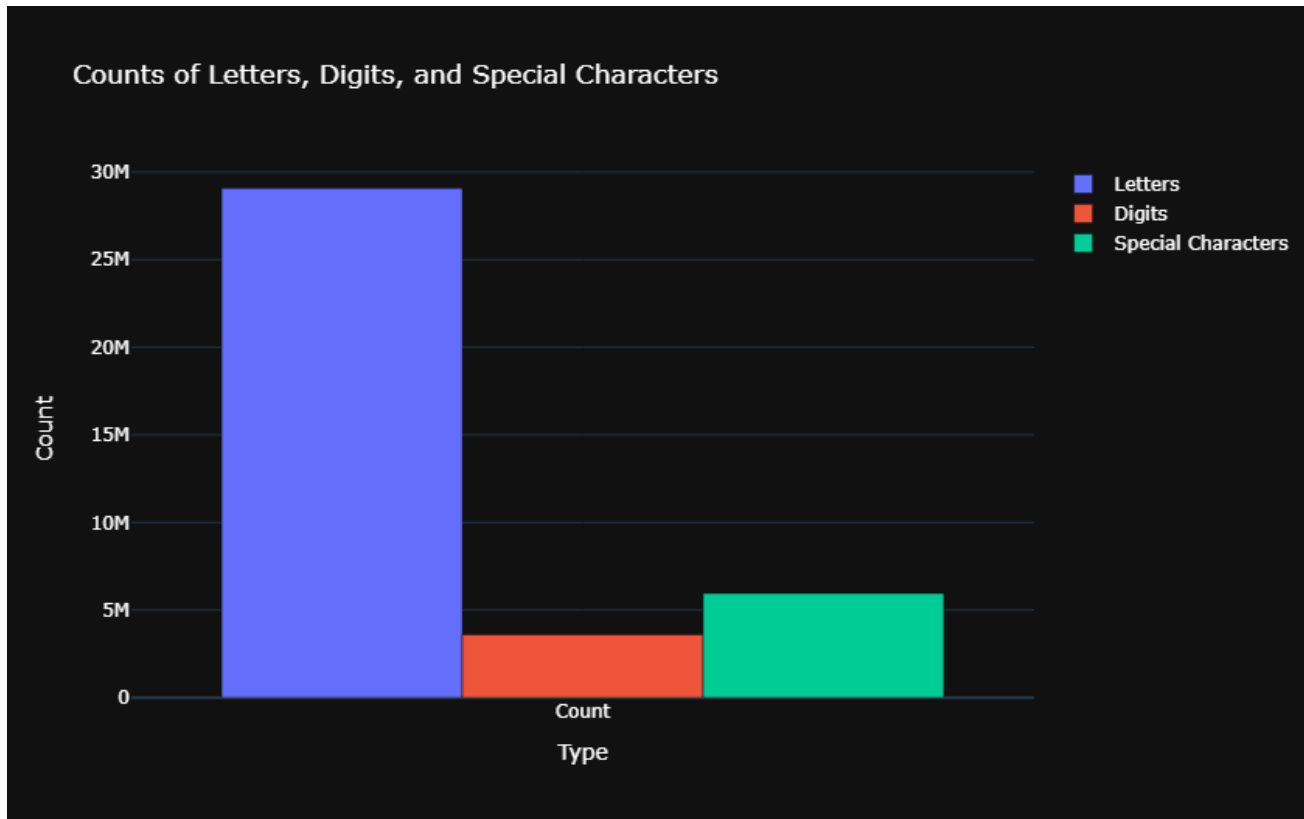
URL Length





Preprocessing-Data Visuals

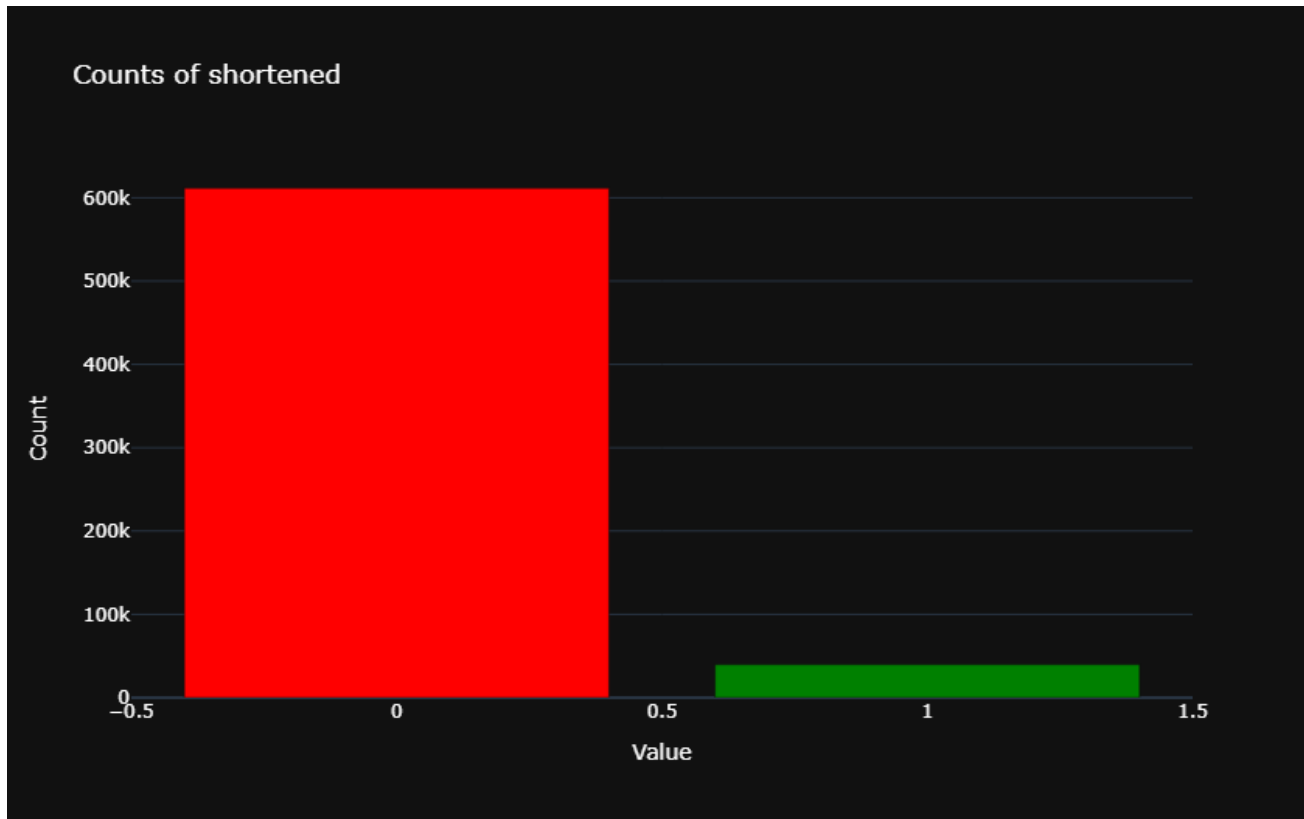
Letters, Digits, Special Characters





Preprocessing-Data Visuals

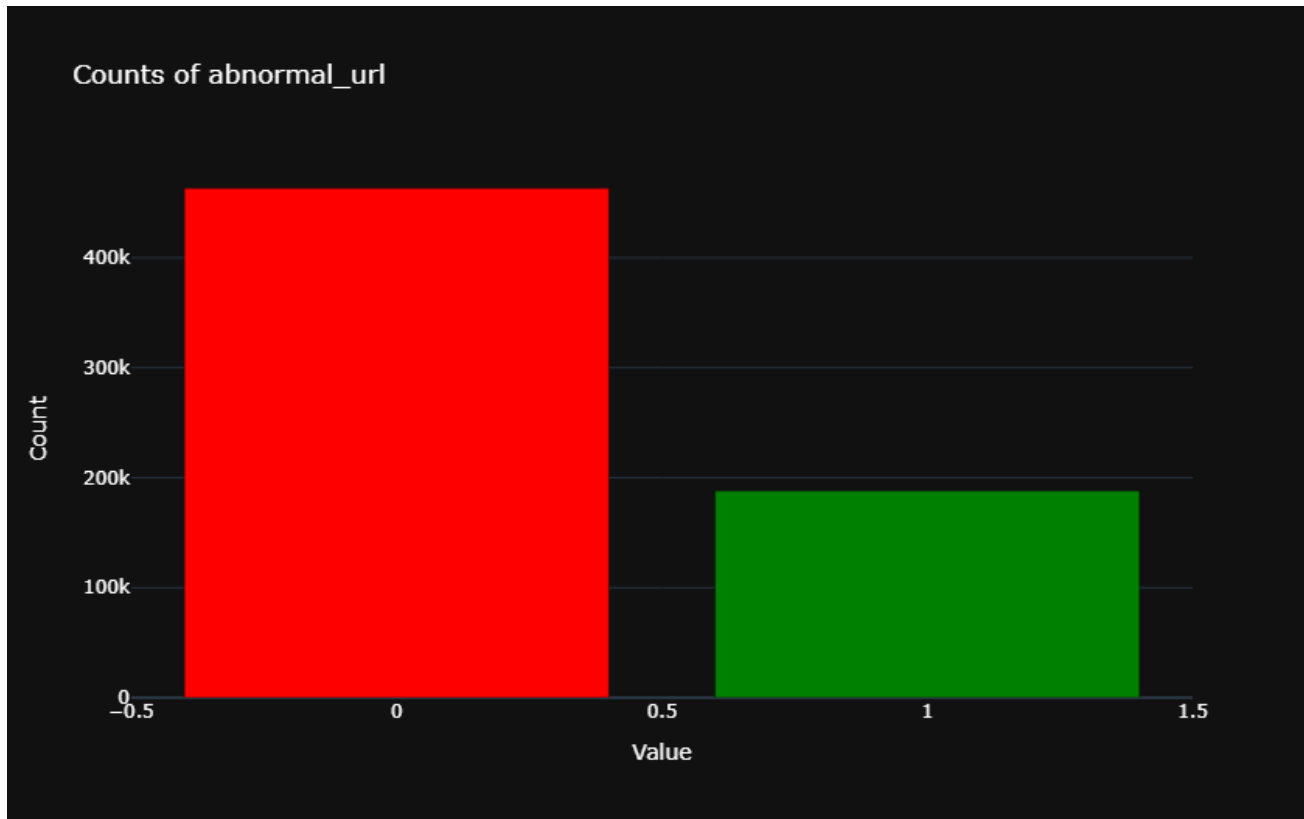
Special Domains





Preprocessing-Data Visuals

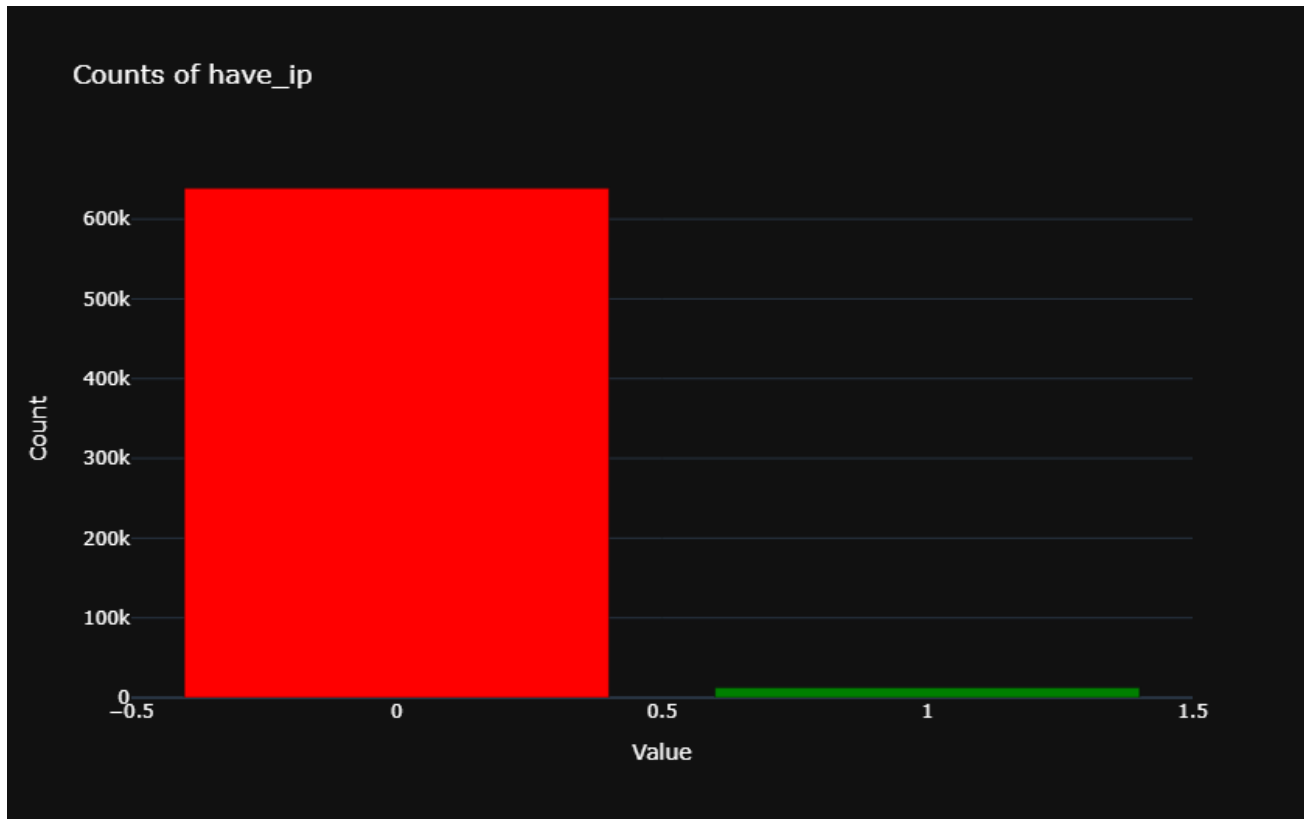
Special Domains





Preprocessing-Data Visuals

Special Domains





BERT Tokenizer

▶ BERT pretrained model from Huggingface

▶ https://huggingface.co/docs/transformers/model_doc/bert

```
from transformers import BertModel, BertTokenizer
import torch

# Load the pre-trained BERT model and tokenizer
model = BertModel.from_pretrained('bert-base-uncased', output_hidden_states=True)
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
```

Downloading (...)lve/main/config.json: 100%  570/570 [00:00<00:00, 14.4kB/s]

Downloading model.safetensors: 100%  440M/440M [00:04<00:00, 68.8MB/s]

Downloading (...)solve/main/vocab.txt: 100%  232k/232k [00:00<00:00, 2.26MB/s]

Downloading (...)okenizer_config.json: 100%  28.0/28.0 [00:00<00:00, 1.04kB/s]



BERT Feature Extraction

- ▶ Output Layer 포함 4개의 Hidden State의 평균
- ▶ Token을 바로 사용할 수는 없음
 - ▶ URL의 길이에 따라 Token의 Shape변동
 - ▶ Hidden State를 사용해 균일한 Shape의 Feature 추출

```
# Define a function to extract features for each transaction
def extract_features(text):
    # Tokenize the text
    input_ids = tokenizer(text, return_tensors="pt", add_special_tokens=True).to(device)['input_ids']
    # Get the hidden states for each token
    with torch.no_grad():
        outputs = model(input_ids)
        hidden_states = outputs[2]
    # Concatenate the last 4 hidden states
    token_vecs = []
    for layer in range(-4, 0):
        token_vecs.append(hidden_states[layer][0])
    # Calculate the mean of the last 4 hidden states
    features = []
    for token in token_vecs:
        features.append(torch.mean(token, dim=0))
    # Return the features as a tensor
    return torch.stack(features)
```



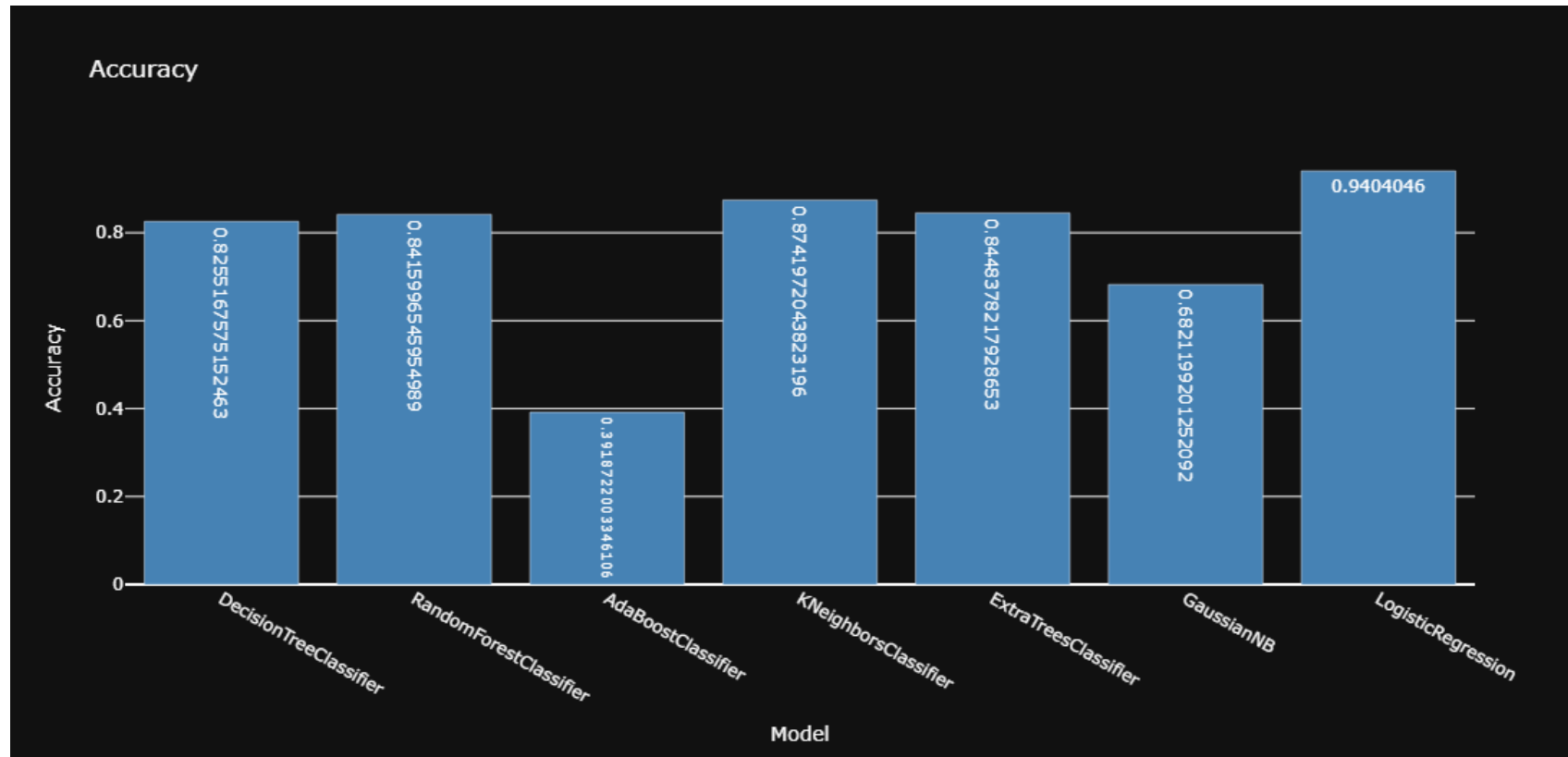
Model

▶ Tested Model

- ▶ Decision Tree Classifier
- ▶ Random Forest Classifier
- ▶ AdaBoost Classifier
- ▶ KNeighbors Classifier
- ▶ Extra Trees Classifier
- ▶ Gaussian Naïve Bayes
- ▶ Logistic Regression
- ▶ Pytorch DNN

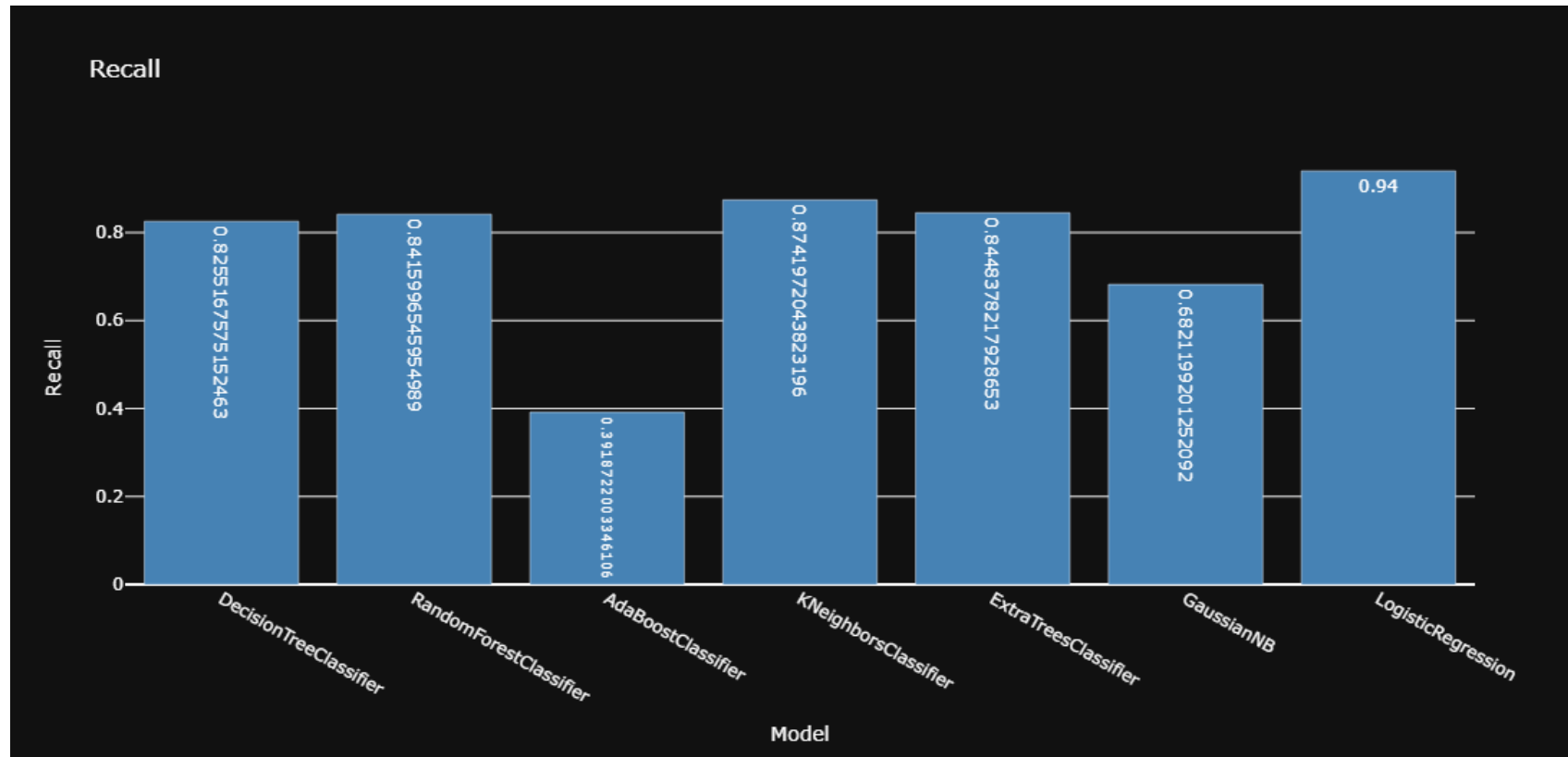


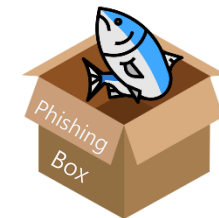
Model-Stats



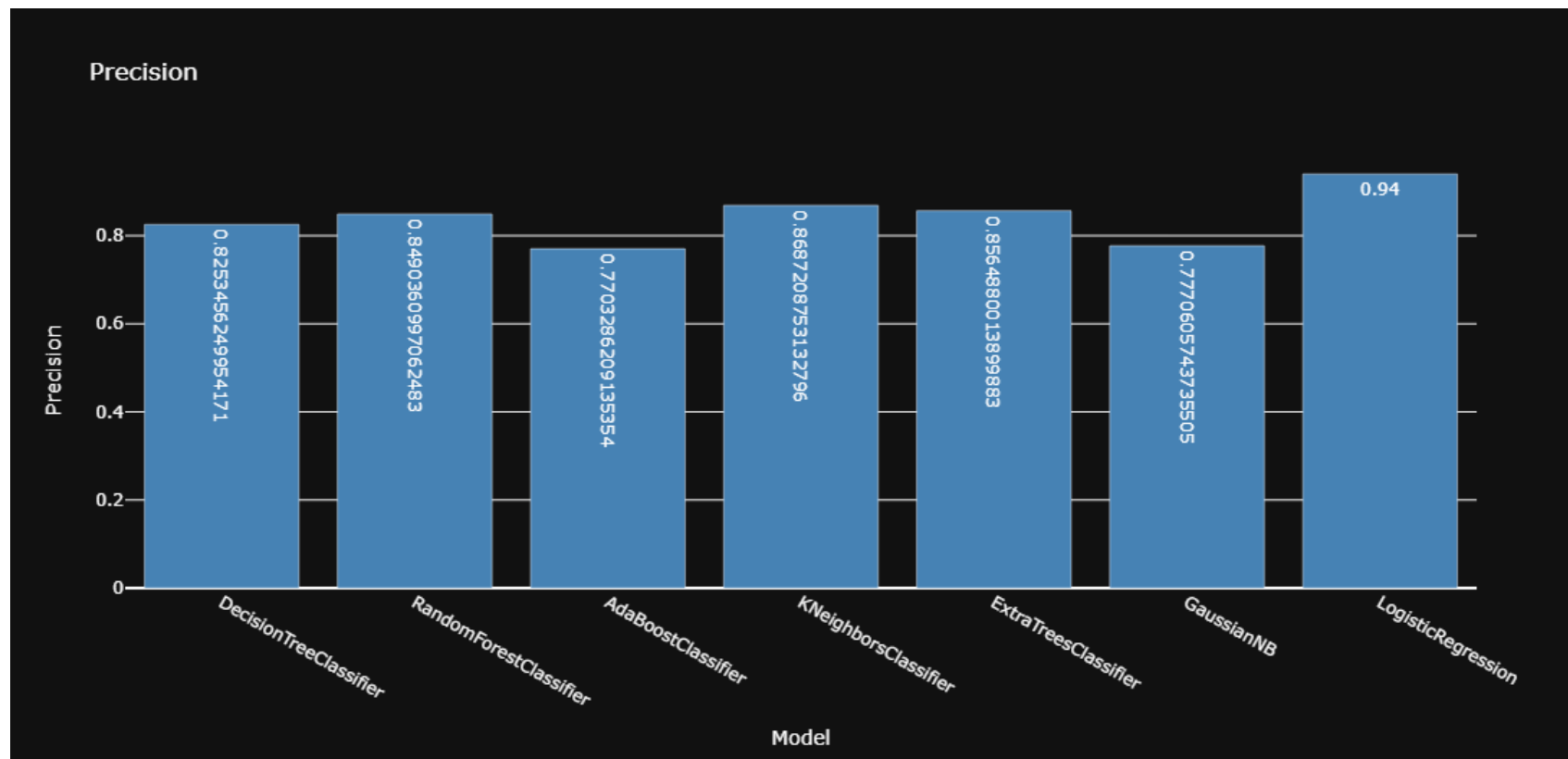


Model-Stats



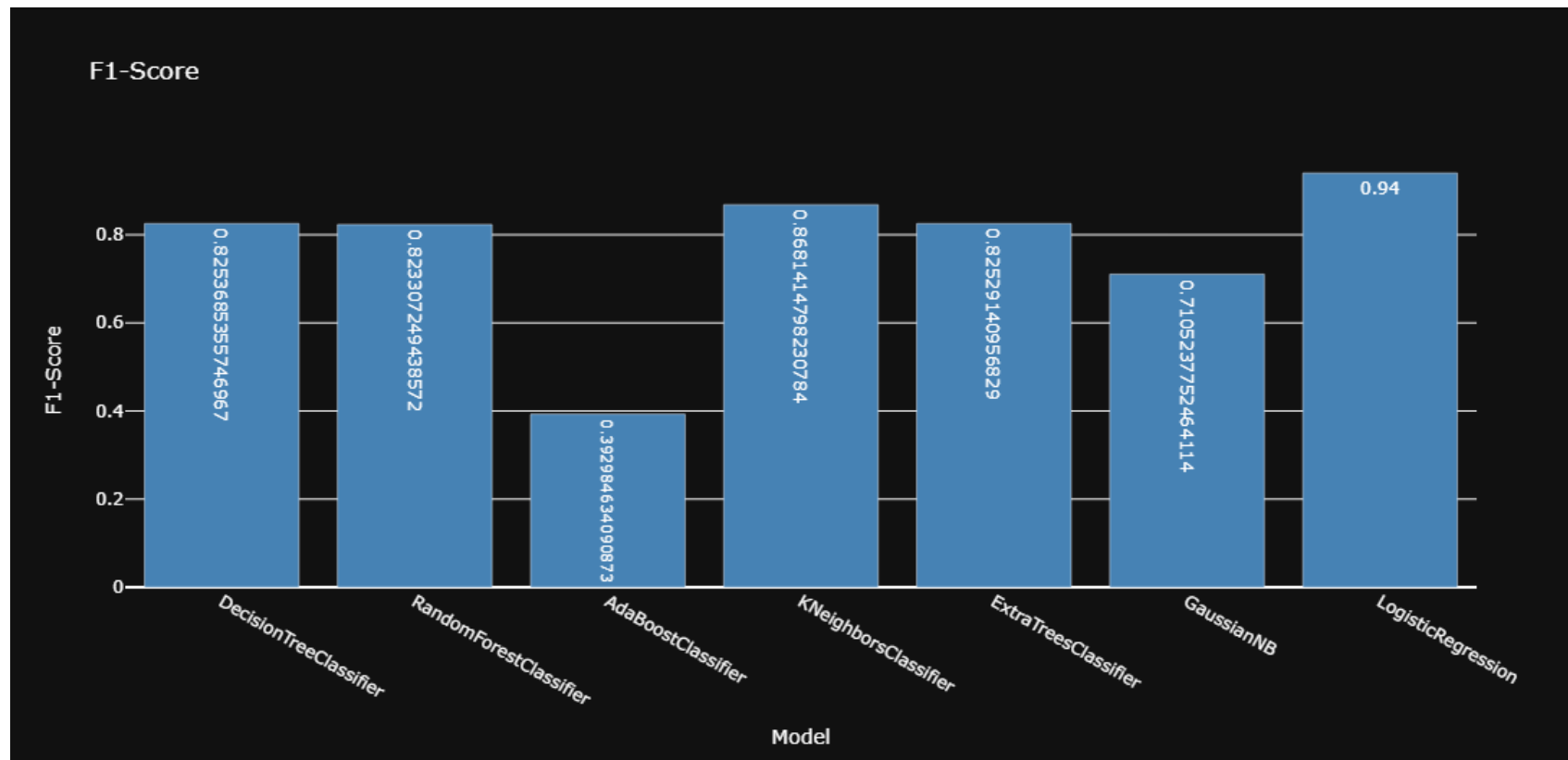


Model-Stats





Model-Stats





Logistic Regression Model Results

```
Epoch: 400 Train loss: 0.038544576130869285
Epoch: 480 Train loss: 0.038288099842882437
Epoch: 490 Train loss: 0.06177155739562803

[ ] val_loss = train(val_data, model, DEVICE, 1, BATCH_SIZE, mode='val')

Epoch: 0 Train loss: 0.07333373421470062

[19] import pickle

#pickle.dump(clf, open(path+'model_bert_logreg_fixed.pkl', 'wb'))

[ ] model_bert = pickle.load(open(path+'model_bert_logreg_fixed.pkl', 'rb'))
model_bert.score(X_test, y_test)

0.9651138608713736

import timeit

print(timeit.timeit("model_bert.score(X_test, y_test)", number=10, globals=globals()))
print(timeit.timeit("train(val_data, model, DEVICE, 1, BATCH_SIZE, mode='val')", number=10, globals=globals()))

1.3830302629999096
0
0
0
0
0
0
0
0
0
0
0
```

0초 오전 12:42에 완료됨



Logistic Regression Model Results

```
print("\nClassification Report:\n", cr)
```

Classification Report:

	precision	recall	f1-score	support
0.0	0.98	0.99	0.98	12745
1.0	0.97	0.98	0.98	2886
2.0	0.93	0.89	0.91	2875
3.0	0.93	0.90	0.91	728
accuracy			0.97	19234
macro avg	0.95	0.94	0.95	19234
weighted avg	0.97	0.97	0.97	19234

```
from sklearn.linear_model import LogisticRegression
```

```
# Train a logistic regression classifier on the training set  
clf = LogisticRegression(max_iter=1000)  
clf.fit(X_train, y_train)
```



Logistic Regression Model Results

```
print("Confusion Matrix:\n", cm)
```

Confusion Matrix:

```
[[12699    14    182    10]
 [   37  2694    34     8]
 [   276    65  2481    21]
 [    14    19    65   615]]
```

```
confusion_matrix(y_train, clf.predict(X_train))
```

```
array([[50575,    75,   650,    21],
       [  111, 11177,    94,    27],
       [  979,   211, 10026,   99],
       [   86,    59,   196,  2548]])
```



DNN vs Logistic

```
import timeit

print(timeit.timeit("model_bert.score(X_test, y_test)")
print(timeit.timeit("train(val_data, model, DEVICE, 1,
```

1.3830302629999096

0

0

0

0

0

0

0

0

0

0

8.19605938299992

```
import torch
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader

class myDataset(Dataset):
    def __init__(self, X, y):
        self.X_values = X
        self.y_values = y
    def __len__(self):
        return self.X_values.shape[0]
    def __getitem__(self, idx):
        item = dict()
        item['features'] = torch.tensor(self.X_values[idx]).float()
        item['target'] = torch.tensor(self.y_values[idx]).long()
        return item

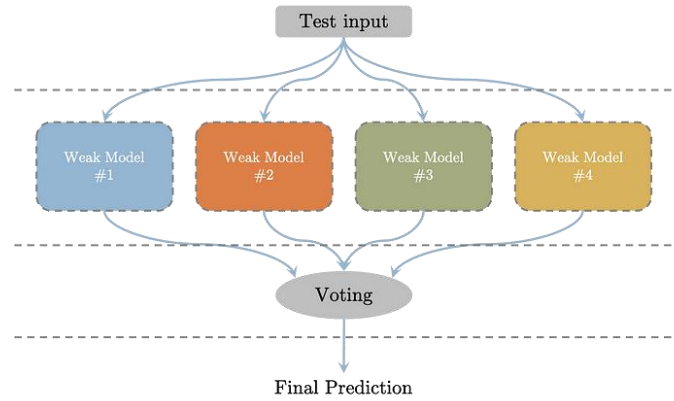
class myDNN(torch.nn.Module):
    def __init__(self, input_size, h1_size, h2_size, output_size=1):
        super().__init__()
        self.fc1 = torch.nn.Linear(input_size, h1_size).to(DEVICE)
        self.fc2 = torch.nn.Linear(h1_size, h2_size).to(DEVICE)
        self.dropout1 = torch.nn.Dropout(0.2)
        self.fc3 = torch.nn.Linear(h2_size, 32).to(DEVICE)
        self.out = torch.nn.Linear(32, output_size).to(DEVICE)
    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.dropout1(x)
        x = F.relu(self.fc3(x))
        x = self.out(x) # CrossEntropy
        return x
```



향후 발전 계획



AWS 유료 구독
서비스로 컴퓨팅
성능 향상



앙상블 기법 등을
적용해 모델의
성능 향상



충분한 사용자 수
확보 이후 창업과
함께 서비스 런칭



감사합니다

AI 보안 기술개발 네트워크반 6조 최종 보고

깃허브 리포지토리

- ▶ https://github.com/yonghoya/Al_Security_Network_Project
- ▶ https://github.com/yonghoya/Al_Security_Network_Project2
- ▶ 데이터셋:
- ▶ <https://www.kaggle.com/datasets/sid321axn/malicious-urls-dataset>

소스 코드 별첨

이하는 발표에 사용되지 못한 소스 코드입니다.

인공지능 관련 코드

▶ 대다수는 코랩에서 작업했습니다.

```
#overhead
from google.colab import drive
drive.mount("/content/MyDrive")
path = "/content/MyDrive/MyDrive/네트워크 과정 프젝/"
```

```
#preprocessing
url_data['url'] = url_data['url'].replace('www.', "", regex=True)
url_data['url'] = url_data['url'].replace('http://', "", regex=True)
url_data['url'] = url_data['url'].replace('https://', "", regex=True)
url_data['url'] = url_data['url'].replace('.html', "", regex=True)
url_data['url'] = url_data['url'].replace('.htm', "", regex=True)
```

```
url_data["url_type"] = url_data["type"].replace({
    'benign':0,
    'defacement':1,
    'phishing':2,
    'malware':3
})
```

```
import re
reg = re.compile('^(.*)\./')
def pri_domain(text):
    temp = reg.findall(text)
    if temp:
        return temp[0]
    else:
        return text
url_data['pri_domain'] = url_data['url'].apply(pri_domain)
```

```
#preprocessing-cont.1
```

```
def count_letters(url):
    num_letters = sum(char.isalpha() for char in url)
    return num_letters
```

```
def count_digits(url):
    num_digits = sum(char.isdigit() for char in url)
    return num_digits
```

```
def count_special_chars(url):
    special_chars = "!@#$%^&*()_+=[{};:,.<>/?'`~|"
    num_special_chars = sum(char in special_chars for char in url)
    return num_special_chars
```

```
url_data['len'] = url_data['url'].apply(len)
url_data['letters_count'] = url_data['url'].apply(lambda x: count_letters(x))
url_data['digits_count'] = url_data['url'].apply(lambda x: count_digits(x))
url_data['special_chars_count'] = url_data['url'].apply(lambda x: count_special_chars(x))
```

인공지능 관련 코드

- ▶ 대다수는 코랩에서 작업했습니다.

```
#preprocessing-cont.2
def has_shortening_service(url):
    pattern = re.compile(r'bit\.ly|goo\.gl|shorte\.st|go2l\.ink|x\.co|ow\.ly|t\.co|tinyurl|tr\.im|is\.gd|cli\.gs|'
        r'yfrog\.com|migre\.me|ff\.im|tiny\.cc|url4\.eu|twit\.ac|su\.pr|twurl\.nl|snipurl\.com|'
        r'short\.to|BudURL\.com|ping\.fm|post\.ly|Just\.as|bkite\.com|snipr\.com|fic\.kr|loopt\.us|'
        r'doiop\.com|short\.ie|kl\.am|wp\.me|rubyurl\.com|om\.ly|to\.ly|bit\.do|t\.co|lnkd\.in|'
        r'db\.tt|qr\.ae|adf\.ly|goo\.gl|bitly\.com|cur\.lv|tinyurl\.com|ow\.ly|bit\.ly|ity\.im|'
        r'q\.gs|is\.gd|po\.st|bc\.vc|twitthis\.com|u\.to|j\.mp|buzurl\.com|cutt\.us|u\.bb|yourls\.org|'
        r'x\.co|prettylinkpro\.com|scrnch\.me|filoops\.info|vzturl\.com|qr\.net|1url\.com|tweez\.me|v\.gd|'
        r'tr\.im|link\.zip\.net')
    match = pattern.search(url)
    return int(bool(match))
url_data['shortened'] =url_data['url'].apply(lambda x: has_shortening_service(x))

from urllib.parse import urlparse
def abnormal_url(url):
    parsed_url = urlparse(url)
    hostname = parsed_url.hostname
    if hostname:
        hostname = str(hostname)
        match = re.search(hostname, url)
        if match:
            return 1
    return 0
url_data['abnormal_url'] =oridf['url'].apply(lambda x: abnormal_url(x))
```

인공지능 관련 코드

- ▶ 대다수는 코랩에서 작업했습니다.

```
#preprocessing-cont.3
def have_ip_address(url):
    pattern = r'((([01]?\d\d?|2[0-4]\d|25[0-5])\.([01]?\d\d?|2[0-4]\d|25[0-5])\.([01]?\d\d?|2[0-4]\d|25[0-5])\.([01]?\d\d?|2[0-4]\d|25[0-5])\s|' \
        r'([01]?\d\d?|2[0-4]\d|25[0-5])\s/)|' \
        r'((([01]?\d\d?|2[0-4]\d|25[0-5])\.([01]?\d\d?|2[0-4]\d|25[0-5])\.([01]?\d\d?|2[0-4]\d|25[0-5])\.([01]?\d\d?|2[0-4]\d|25[0-5])\s|' \
        r'([01]?\d\d?|2[0-4]\d|25[0-5])\s/)|' \
        r'((0x[0-9a-fA-F]{1,2})\. (0x[0-9a-fA-F]{1,2})\. (0x[0-9a-fA-F]{1,2})\. (0x[0-9a-fA-F]{1,2})\s/)' \
        r'(?:[a-fA-F0-9]{1,4}:){7}[a-fA-F0-9]{1,4}|' \
        r'[0-9]+(?:\.[0-9]+){3}:[0-9]+)' \
        r'((?:(?:\d|[01]?\d\d|2[0-4]\d|25[0-5])\.){3}(?:25[0-5]|2[0-4]\d|[01]?\d\d|\d)(?:\s/\s\d{1,2})?)'

    match = re.search(pattern, url)
    if match:
        return 1
    else:
        return 0
url_data['have_ip'] =url_data['url'].apply(lambda x: have_ip_address(x))

url_data.to_csv("/content/MyDrive/MyDrive/네트워크 과정 프젝/preprocessed_fixed.csv")
```

인공지능 관련 코드

▶ 대다수는 코랩에서 작업했습니다.

```
#AI_preprocessing
!pip -q install transformers

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
from statsmodels.stats.outliers_influence import variance_inflation_factor

from sklearn.model_selection import cross_val_score

url_data = pd.read_csv(path+'preprocessed_fixed.csv')

from transformers import BertModel, BertTokenizer
import torch

# Load the pre-trained BERT model and tokenizer
model = BertModel.from_pretrained('bert-base-uncased', output_hidden_states=True)
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
device = "cuda:0" if torch.cuda.is_available() else "cpu"
```

```
#AI_preprocessing_cont.1
# Define a function to extract features for each transaction
def extract_features(text):
    # Tokenize the text
    input_ids = tokenizer(text, return_tensors="pt", add_special_tokens=True).to(device)['input_ids']
    # Get the hidden states for each token
    with torch.no_grad():
        outputs = model(input_ids)
        hidden_states = outputs[2]
    # Concatenate the last 4 hidden states
    token_vecs = []
    for layer in range(-4, 0):
        token_vecs.append(hidden_states[layer][0])
    # Calculate the mean of the last 4 hidden states
    features = []
    for token in token_vecs:
        features.append(torch.mean(token, dim=0))
    # Return the features as a tensor
    return torch.stack(features)
```

인공지능 관련 코드

▶ 대다수는 코랩에서 작업했습니다.

```
#AI preprocessing_cont.2
dftest = url_data.drop(['Unnamed: 0', 'type'], axis=1)

from sklearn.model_selection import train_test_split, cross_val_score, cross_val_predict

x = dftest.drop(columns=['url_type','bw'])
y = dftest['url_type']
_,x_data,_,y_data = train_test_split(x,y,test_size=0.60, random_state=42)#68

features = []
for i in range(len(x_data)):
    if len(x_data.iloc[i]['url']) < 512:
        features.append(extract_features(x_data.iloc[i]['url']))
    else:
        features.append(extract_features(x_data.iloc[i]['pri_domain'][:511]))
    if i%10000==0:
        print(i)
# Concatenate the features and convert to a numpy array
features = torch.cat(features).cpu().numpy()
```

```
#AI preprocessing_cont.3

# features is a 2D numpy array of size 4000x768
# labels is a 1D numpy array of size 1000
# reshape the feature array to size 1000x(768*4)

features_resaped = features.reshape((len(x_data), -1))
temp = x_data[['len','letters_count','digits_count', 'special_chars_count',
'shortened', 'abnormal_url', 'have_ip']].values.reshape(-1, 7)
#temp = np.hstack((temp, y_data.values.reshape(-1,1)))

dataset = np.hstack((features_resaped, temp))
dataset = np.hstack((dataset, y_data.values.reshape(-1,1)))

# Split the data into training and testing sets
train_data, test_data = train_test_split(dataset, test_size=0.2, random_state=42)

# Convert the training and testing sets back into separate feature and label arrays
X_train, y_train = train_data[:, :-1], train_data[:, -1]
X_test, y_test = test_data[:, :-1], test_data[:, -1]
```

인공지능 관련 코드

▶ 대다수는 코랩에서 작업했습니다.

```
#AI
from sklearn.linear_model import LogisticRegression

# Train a logistic regression classifier on the training set
clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)

#AI scoring

# Evaluate the classifier on the testing set
score = clf.score(X_test, y_test)
score
from sklearn.metrics import confusion_matrix, classification_report
y_pred = clf.predict(X_test)
# Generate the confusion matrix and classification report
cm = confusion_matrix(y_test, y_pred)
cr = classification_report(y_test, y_pred)

print("Confusion Matrix:\n", cm)
print("\nClassification Report:\n", cr)
```

인공지능 관련 코드

▶ 대다수는 코랩에서 작업했습니다.

```
#Pytorch DNN
import torch
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader

class myDataset(Dataset):
    def __init__(self, X, y):
        self.X_values = X
        self.y_values = y
    def __len__(self):
        return self.X_values.shape[0]
    def __getitem__(self, idx):
        item = dict()
        item['features'] = torch.tensor(self.X_values[idx]).float()
        item['target'] = torch.tensor(self.y_values[idx]).long()
        return item

class myDNN(torch.nn.Module):
    def __init__(self, input_size, h1_size, h2_size, output_size=1):
        super().__init__()
        self.fc1 = torch.nn.Linear(input_size, h1_size).to(DEVICE)
        self.fc2 = torch.nn.Linear(h1_size, h2_size).to(DEVICE)
        self.dropout1 = torch.nn.Dropout(0.2)
        self.fc3 = torch.nn.Linear(h2_size, 32).to(DEVICE)
        self.out = torch.nn.Linear(32, output_size).to(DEVICE)
    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.relu(self.fc2(x))
        x = self.dropout1(x)
        x = F.relu(self.fc3(x))
        x = self.out(x) # CrossEntropy
        return x
```

인공지능 관련 코드

▶ 대다수는 코랩에서 작업했습니다.

```
#Pytorch DNN_cont.1
def train(dataset, model, device, epochs, batch_size, mode='train', lr=0.01):
    optimizer = torch.optim.SGD(model.parameters(), lr=lr)
    loss_fn = torch.nn.CrossEntropyLoss()
    train_loss_history = []
    val_loss_history = []
    loader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
    for e in range(int(epochs)):
        if mode=='train':
            model.train()
            train_loss = 0
            for batch in loader:
                optimizer.zero_grad()
                inputs=batch['features'].to(device)
                labels=batch['target'].to(device)
                preds = model(inputs)
                loss = loss_fn(preds, labels)
                loss.backward()
                optimizer.step()
                train_loss += loss.item()
            train_loss_history.append(train_loss/len(loader))
        if mode=='val':
            model.eval()
            train_loss = 0
            for batch in loader:
                optimizer.zero_grad()
                inputs=batch['features'].to(device)
                labels=batch['target'].to(device)
                preds = model(inputs)
                loss = loss_fn(preds, labels)
                train_loss += loss.item()
            train_loss_history.append(train_loss/len(loader))
        if e%10 == 0:
            #print("Epoch: {} Train loss: {}".format(e, train_loss/len(loader)))
            pass
        print(e)
    return train_loss_history
```

인공지능 관련 코드

▶ 대다수는 코랩에서 작업했습니다.

```
#Pytorch DNN_cont.2
EPOCHS=1000
BATCH_SIZE=512
DEVICE=('cuda' if torch.cuda.is_available() else 'cpu')

train_data = myDataset(X_train, y_train)
val_data = myDataset(X_test, y_test)

input_size = (X_train.shape[1])
h1_size = input_size*2

model = myDNN(input_size, h1_size, 128, 4)

train_loss = train(train_data, model, DEVICE, EPOCHS, BATCH_SIZE, mode='train')
val_loss = train(val_data, model, DEVICE, 1, BATCH_SIZE, mode='val')
```